1. Proposed Research Topic 1: Typing Pattern Detector for Developer Break Reminders

Problem Statement

Software developers often work for long periods without breaks, leading to fatigue, reduced productivity, and increased coding errors. Although IDEs track keystrokes and editing actions, they do not analyze typing behavior to detect signs of developer fatigue such as slower typing speed or increased correction frequency.

Research Objective

Develop a VS Code plugin that monitors typing speed and editing patterns to detect potential developer fatigue and recommend short breaks when fatigue indicators appear.

Research Questions

- Can typing speed and error frequency predict when developers are becoming fatigued?
- Do break reminders improve coding performance or reduce bugs?
- Which typing behavior patterns most accurately indicate developer fatigue?

System Overview

VS Code extension consisting of:

- **Typing Tracker:** Measures characters per minute and backspaces per minute
- **Fatigue Scoring System:** Detects significant drops in typing speed combined with increased errors
- **Break Notification Module:** Displays break suggestions with a short timer when fatigue thresholds are reached

Implementation Plan

Phase 1: Develop base VS Code extension framework (2 days)
Phase 2: Implement typing event tracking (3 days)
Phase 3: Design fatigue scoring algorithm (2 days)
Phase 4: Implement break reminder notifications (2 days)
Phase 5: Conduct initial testing with volunteer developers (3 days)

Evaluation Methodology

- Recruit approximately **10 developers** for testing
- Participants complete coding tasks with and without the plugin

- Measure:
    - Typing speed changes
    - Number of bugs produced
    - User feedback on break reminders

Expected Contributions

- A lightweight **open-source VS Code plugin for developer well-being**
- Empirical evidence linking typing behavior to developer fatigue
- A simple fatigue detection model based on typing metrics

Feasibility (30-Day Scope)

Requires only a laptop and VS Code development tools.
The plugin uses existing **VS Code APIs**, and the fatigue model relies on simple statistical rules rather than machine learning.

---

2. Proposed Research Topic 2: Code Complexity Warning Tool for New Developers

Problem Statement

Beginner programmers often struggle with overly complex code structures such as deeply nested logic or excessively long functions. Current IDEs highlight syntax errors but rarely warn developers when code becomes unnecessarily complex and difficult to maintain.

Research Objective

Develop a VS Code extension that identifies potentially complex code structures and provides visual warnings and suggestions to help novice developers simplify their code.

Research Questions

- Can simple code metrics effectively detect complex or difficult-to-understand code?
- Do complexity warnings help junior developers write cleaner code?
- Which complexity indicators are most useful for identifying problematic code?

System Overview

VS Code plugin including:

- **Code Complexity Analyzer:** Detects deep nesting, long functions, and other complexity indicators
- **Visual Highlighting:** Colors complex code segments to draw developer attention
- **Simplification Suggestions:** Provides recommendations for reducing complexity

Implementation Plan

Phase 1: Implement code parsing and complexity detection rules (3 days)
Phase 2: Develop visual highlighting system within VS Code (2 days)
Phase 3: Implement suggestion system for code simplification (3 days)
Phase 4: Conduct testing with junior developers (4 days)

Evaluation Methodology

- Recruit **junior and experienced developers**
- Assign coding tasks with the plugin enabled and disabled
- Measure:
    - Task completion time
    - Number of coding errors
    - User feedback on complexity warnings

Expected Contributions

- Open-source **code complexity detection tool for beginners**
- Insights into common sources of difficulty for junior developers
- Simple rule-based framework for detecting complex code structures

Feasibility (30-Day Scope)

The project uses **JavaScript/TypeScript and existing VS Code extension templates**, making implementation achievable within a short timeframe. Testing can be performed with student developers.

---

3. Proposed Research Topic 3: Typing Fatigue Detector for Developer Break Optimization

topic 1 uses a generic threshold for everyone, topic 3 personalizes the baseline per user which makes the fatigue detection more accurate, and it also tracks whether breaks actually improve performance which topic 1 doesn't do

so topic 3 needs a bit of actual data storage and statistical comparison per user. topic 1 doesn't need to remember anything about you.

Problem Statement

Developer productivity often decreases due to cognitive fatigue during extended coding sessions. One observable indicator of fatigue is a decline in typing speed accompanied by increased correction behavior. However, modern development environments do not monitor such behavioral signals to recommend optimal break timing.

Research Objective

Develop a VS Code plugin that monitors typing speed and correction behavior to estimate developer fatigue and suggest break timing that improves productivity and reduces coding errors.

Research Questions

- Does a significant drop in typing speed predict increased coding errors?
- Can timely break reminders restore typing performance to baseline levels?
- Does the ratio of corrections to typing actions indicate cognitive fatigue?

System Overview

VS Code plugin consisting of four main modules:

- **Typing Monitor:** Records typing speed and backspace frequency
- **Fatigue Score Calculator:** Computes a fatigue score based on typing behavior
- **Break Notification Engine:** Triggers alerts when fatigue thresholds are exceeded
- **Recovery Tracker:** Measures whether typing speed improves after breaks

Implementation Plan

Phase 1: Implement VS Code extension and typing event tracking (3 days)
Phase 2: Develop baseline typing speed calculator for each user (3 days)
Phase 3: Implement fatigue scoring algorithm and alert system (3 days)
Phase 4: Conduct user study with developer participants (5 days)
Phase 5: Analyze collected data and prepare research documentation (7 days)

Evaluation Methodology

- Conduct a controlled experiment with **15 developers**
- Compare two scenarios:
    - Standard VS Code environment
    - VS Code with fatigue detection plugin
- Metrics collected:
    - Bugs per hour
    - Typing speed recovery after breaks
    - Subjective workload survey (NASA-TLX)

Expected Contributions

- A lightweight **developer fatigue monitoring tool**
- Dataset of typing behavior patterns related to coding fatigue
- Evidence on how break timing influences developer productivity

Feasibility (30-Day Scope)

The project requires only a personal computer and standard development tools.
Implementation relies on **VS Code APIs and simple mathematical models**, avoiding the need for complex machine learning systems.

4. Proposed Research Topic 4: Human-in-the-Loop Labeling for Small Dataset Improvement

Problem Statement

Machine learning models often require large labeled datasets, but labeling data is expensive and time-consuming. Many real-world projects start with very small datasets, and it is unclear whether labeling the **most uncertain examples** actually improves model performance compared to labeling random samples.

Research Objective

Build a simple machine learning pipeline that compares **uncertainty-based labeling vs random labeling** and measure how quickly model accuracy improves when new labels are added.

Research Questions

- Does selecting uncertain samples improve model accuracy faster than random labeling?
- How many additional labeled examples are needed to significantly improve performance?
- Do uncertainty-based methods consistently outperform random sampling across different datasets?

System Overview

Python experiment pipeline with:

- Small labeled dataset (initial training set)
- Large unlabeled data pool
- Baseline classifier (Logistic Regression / Random Forest)

- Uncertainty sampling method (least confidence or entropy)
- Iterative labeling simulation
- Accuracy tracking and visualization

Implementation Plan

Phase 1: Dataset preparation and baseline model (3 days)
Phase 2: Implement uncertainty scoring method (3 days)
Phase 3: Simulate iterative labeling process (4 days)
Phase 4: Implement random labeling comparison (3 days)
Phase 5: Generate plots and run experiments (4 days)

Evaluation Methodology

- Use small public datasets (e.g., spam detection, sentiment analysis)
- Train models starting with **10–20 labeled samples**
- Iteratively add new labeled examples
- Compare accuracy improvements for:
    - Uncertainty-based labeling
    - Random labeling
- Plot **accuracy vs number of labeled samples**

Expected Contributions

- Simple demonstration of **human-in-the-loop labeling strategies**
- Experimental comparison of uncertainty vs random sampling
- Reproducible code for teaching data-centric ML concepts

Feasibility (30-Day Scope)

Only requires a laptop and Python libraries (Scikit-learn, Matplotlib).
Datasets are publicly available and no complex deep learning training is required.

---

5. Proposed Research Topic 5: Explainable AI for Cybersecurity Threat Detection

Problem Statement

Cybersecurity systems often use machine learning models to detect threats such as phishing emails or malicious network traffic. However, these models act like black boxes, making it difficult for analysts to understand **why a specific event was classified as a threat**.

Research Objective

Develop a cybersecurity classification model and evaluate how **Explainable AI (XAI) techniques** help interpret the model's decisions and identify suspicious patterns.

Research Questions

- Can XAI methods clearly explain why a cybersecurity event is flagged as malicious?
- Which explanation techniques provide the most understandable insights for analysts?
- Can explanations help identify biases or misleading features in the model?

System Overview

Cybersecurity classification system with:

- Public phishing or intrusion detection dataset
- Baseline classifier (Random Forest or Logistic Regression)
- XAI explanation tools:
    - LIME
    - SHAP
- Visualization of important features influencing predictions

Implementation Plan

Phase 1: Dataset preprocessing and baseline classifier (4 days)
Phase 2: Implement phishing or intrusion detection model (4 days)
Phase 3: Integrate XAI tools (LIME / SHAP) (4 days)
Phase 4: Generate explanation visualizations (4 days)
Phase 5: Evaluate explanations with simple case studies (4 days)

Evaluation Methodology

- Train a model to classify phishing emails or malicious network traffic
- Apply XAI methods to explain predictions
- Compare explanations across different models
- Analyze whether explanations highlight meaningful cybersecurity features

Expected Contributions

- Case study demonstrating **interpretable AI in cybersecurity**
- Comparison of explanation methods for threat detection models
- Insights into how XAI can help analysts understand model decisions

Feasibility (30-Day Scope)

Uses publicly available datasets and lightweight ML models.
XAI tools like **LIME and SHAP** have ready-to-use Python libraries, allowing implementation within a short research timeline.

1. https://arxiv.org/html/2501.00277
2. https://www.researchgate.net/publication/392392731_Explainable_AI_XAI_for_Cybersecurity_Decision-Making_Using_SHAP_and_LIME_for_Transparent_Threat_Detection
3. https://www.mdpi.com/2076-3417/15/13/7329
4. https://www.researchgate.net/publication/378352670_Explainable_AI_for_Intrusion_Detection_Systems_LIME_and_SHAP_Applicability_on_Multi-Layer_Perceptron

most research targets enterprise level systems with heavy models and massive datasets. we're specifically studying whether LIME and SHAP provide useful explanations on lightweight models like Random Forest,Logistic Regression which is what small teams actually use.(for topic 4 and 5)

Research Topics
1)Simulation of BB84 Quantum Key Distribution: Analyzing Eavesdropper Detection Rates Under Varying Noise Conditions
2)Comparative Analysis of Random Forest and XGBoost for Real-Time Network Intrusion Detection on NSL-KDD
3)Performance Comparison of CNN Architectures for Image Classification
4)Comparison Evalutaion of BERT vs RoBerta vs DistilBERT(needs careful GPU management and google colab access hai to very easy)

5)Optimizing RAG for question answering using small language models (Langchain ya LLM exposure hai toh achi hai topic)(also the easiest one when compared to the other 3)

6)Solving the Traveling salesman problem using Quantum Approximate optimization algorithm (QAOA) and comparing with classical algorithms (Debugging is risky)

7) Montioring and ovservability of Microservives using Docker, Prometheus and Grafana(we can reduce the devops tools)

(NOUMAN)

Proposed Research Topic 1: Fine-tuning vs Retrieval-Augmented Generation for Domain Adaptation in Small Language Models

Problem Statement

Large Language Models (LLMs) are often adapted to domain-specific tasks such as legal analysis, medical question answering, or software documentation assistance. Two major approaches for domain adaptation are model fine-tuning and Retrieval-Augmented Generation (RAG). Fine-tuning modifies the model weights using domain data, while RAG retrieves external documents during inference to provide relevant context. While both approaches are widely used, there is limited systematic research comparing their performance specifically on small language models under limited computational resources. Developers often struggle to determine which approach provides better performance in terms of accuracy, latency, and cost.

Research Objective

Develop an experimental framework to compare fine-tuning and RAG approaches for domain adaptation in small language models and evaluate their effectiveness across multiple performance metrics.

Research Questions

- Does fine-tuning improve domain knowledge more effectively than RAG for small language models?
- How do RAG systems compare to fine-tuned models in terms of latency and computational efficiency?
- Can hybrid approaches combining RAG with lightweight fine-tuning improve performance?

System Overview

Experimental domain-adapted QA system consisting of:

Domain Dataset Module
Collects domain-specific documents such as legal texts, research articles, or technical documentation.

Fine-tuning Module
Applies parameter-efficient fine-tuning techniques such as LoRA to adapt a small language model.

RAG Pipeline
Retrieves relevant documents from a vector database to provide contextual input to the model.

Hybrid System
Combines retrieval with fine-tuned models to evaluate combined performance.

Evaluation Module
Compares the approaches using accuracy, latency, and resource consumption.

Implementation Plan

Phase 1: Collect and preprocess domain dataset – 4 days
Phase 2: Implement baseline small language model – 3 days
Phase 3: Implement parameter-efficient fine-tuning pipeline – 5 days
Phase 4: Build RAG system with vector database – 5 days
Phase 5: Evaluate fine-tuning vs RAG vs hybrid approaches – 5 days

Evaluation Methodology

Use a standardized query dataset within the chosen domain.

Metrics measured:

- Answer accuracy
- Response latency
- Computational cost
- Hallucination frequency

Compare performance of fine-tuned models, RAG systems, and hybrid approaches.

Expected Contributions

A comparative study of two major domain adaptation paradigms for small language models.
Insights into the trade-offs between fine-tuning and retrieval-based approaches.
Guidelines for selecting domain adaptation strategies for resource-constrained environments.

Feasibility (30-Day Scope)

The system can be implemented using open-source small language models and parameter-efficient fine-tuning methods. Public domain datasets and existing RAG frameworks can be used without requiring large-scale computational resources.

Proposed Research Topic 2: Hallucination Detection in Large Language Model Outputs using Confidence Scoring

Problem Statement

Large Language Models often generate responses that appear plausible but contain incorrect or fabricated information, a phenomenon known as hallucination. This issue poses significant challenges in applications such as healthcare, education, and legal systems where factual accuracy is critical. Existing approaches for hallucination detection often require external knowledge bases or human verification. There is a need for lightweight and automated methods that can identify potential hallucinations directly from model outputs.

Research Objective

Develop a confidence-based hallucination detection mechanism that estimates the reliability of language model outputs using token probability and uncertainty metrics.

Research Questions

- Can token probability distributions be used to detect hallucinated responses?
- What confidence scoring methods best correlate with factual accuracy?
- Can lightweight hallucination detection models improve reliability in practical LLM deployments?

System Overview

LLM evaluation framework consisting of:

LLM Response Generator
Produces answers for user queries using a language model.

Confidence Scoring Module
Calculates token-level probabilities and entropy scores.

Hallucination Detection Module
Flags responses with low confidence or high uncertainty.

Benchmark Dataset Module
Uses known datasets containing factual and hallucinated responses.

Evaluation Module
Measures detection performance.

Implementation Plan

Phase 1: Collect benchmark datasets containing factual and hallucinated responses – 4 days
Phase 2: Implement LLM response generation pipeline – 3 days
Phase 3: Develop confidence scoring algorithms – 5 days
Phase 4: Implement hallucination detection thresholds – 4 days
Phase 5: Conduct experiments and analyze detection accuracy – 5 days

Evaluation Methodology

Test the system using established hallucination benchmark datasets.

Metrics:

- Hallucination detection accuracy
- Precision and recall
- False positive rate
- Detection latency

Compare the proposed confidence-scoring method with baseline detection techniques.

Expected Contributions

A lightweight hallucination detection method based on token confidence scoring.
Empirical evaluation of confidence metrics for detecting unreliable LLM outputs.
A practical framework for improving reliability in LLM-based applications.

Feasibility (30-Day Scope)

The proposed method relies on existing language model outputs and probability scores generated during inference. No model retraining is required, making the approach feasible with moderate computing resources.

Proposed Research Topic 3: Systematic Evaluation of Chunking Strategies in Retrieval-Augmented Generation Pipelines

Problem Statement

Retrieval-Augmented Generation systems rely on dividing documents into smaller segments before storing them in vector databases for retrieval. The chunking strategy used during this preprocessing stage significantly affects retrieval quality and answer accuracy. However, many implementations rely

on simple fixed-length chunking without evaluating alternative strategies such as sliding windows or semantic segmentation. Despite its importance, there is limited empirical research systematically comparing chunking methods across multiple RAG pipelines.

Research Objective

Conduct a systematic evaluation of different document chunking strategies and analyze their impact on retrieval effectiveness and answer accuracy in RAG systems.

Research Questions

- How do different chunking strategies influence retrieval relevance in RAG systems?
- Which chunking strategy produces the highest answer accuracy for question-answering tasks?
- What trade-offs exist between chunk size, retrieval latency, and context coverage?

System Overview

Experimental RAG system consisting of:

Document Preprocessing Module
Processes and prepares documents for chunking.

Chunking Strategy Module
Implements multiple chunking methods including fixed-length, sliding window, and semantic chunking.

Embedding Generation Module
Generates vector embeddings for document chunks.

Vector Database
Stores embeddings and retrieves relevant chunks.

Answer Generation Module
Generates responses using retrieved context.

Implementation Plan

Phase 1: Collect document dataset and preprocess text – 3 days
Phase 2: Implement baseline fixed-length chunking – 3 days
Phase 3: Implement sliding window and semantic chunking strategies – 5 days
Phase 4: Integrate chunking strategies with RAG pipeline – 4 days
Phase 5: Conduct experiments and analyze results – 5 days

Evaluation Methodology

Run experiments with a consistent query set.

Metrics measured:

- Retrieval relevance score
- Answer accuracy
- Context coverage
- Response latency

Compare performance across different chunking strategies.

Expected Contributions

A systematic empirical comparison of chunking strategies used in RAG systems.
Insights into the relationship between chunking techniques and retrieval performance.
Guidelines for selecting chunking strategies in knowledge-based AI systems.

Feasibility (30-Day Scope)

All chunking strategies rely on lightweight text processing methods and can be implemented using existing RAG frameworks and open-source NLP tools.

Proposed Research Topic 4: Comparative Evaluation of Embedding Models for Domain-Specific Semantic Search

Problem Statement

Embedding models are widely used for semantic search and information retrieval in modern AI systems. Several embedding models such as BGE, E5, and Instructor models are commonly used to represent textual information as vector embeddings. While these models demonstrate strong performance on general datasets, their effectiveness on domain-specific corpora remains less studied. Developers often select embedding models without clear evidence of their performance differences in specialized domains such as legal, academic, or technical documentation.

Research Objective

Evaluate and compare multiple embedding models for semantic search tasks in a domain-specific dataset and identify the most effective models for specialized knowledge retrieval.

Research Questions

- How do different embedding models perform on domain-specific semantic search tasks?
- Do general-purpose embedding models perform well in specialized domains?
- What trade-offs exist between retrieval accuracy and computational efficiency?

System Overview

Semantic search system consisting of:

Document Corpus Module
Collects domain-specific documents.

Embedding Generation Module
Generates embeddings using multiple embedding models.

Vector Database
Stores embeddings and supports similarity search.

Query Processing Module
Processes user queries and retrieves relevant documents.

Evaluation Module
Measures retrieval accuracy and system efficiency.

Implementation Plan

Phase 1: Collect and preprocess domain dataset – 3 days
Phase 2: Generate embeddings using multiple models – 5 days
Phase 3: Store embeddings in vector database – 3 days
Phase 4: Implement semantic search pipeline – 4 days
Phase 5: Conduct evaluation and benchmarking experiments – 5 days

Evaluation Methodology

Use a standardized query set for evaluation.

Metrics:

- Retrieval accuracy
- Precision@k and Recall@k
- Query latency
- Memory usage

Compare performance of multiple embedding models.

Expected Contributions

A domain-specific benchmark comparing popular embedding models for semantic search.
Insights into how embedding models perform across specialized datasets.
Practical guidelines for selecting embeddings in domain-specific AI applications.

Feasibility (30-Day Scope)

Embedding models are available through open-source libraries and can be evaluated using moderate-sized datasets. Experiments can be performed using standard computing hardware

---

TOPIC 5- Testing Chat AI's Ability to Understand People's Thoughts (Theory of Mind in LLMs)

Domain: Deep Learning / Natural Language Processing (NLP)

Sub-area: Large Language Model Evaluation + Social Reasoning

Problem Statement

Chat AIs like Llama and GPT can solve stories about what people think or believe - sometimes as good as 7-year-old kids. But they fail hard when stories change slightly: different names, extra confusing details, longer text, or tricky wording. Current tests use fixed stories that AIs might memorize instead of really understanding people's thoughts. No simple tool exists to automatically make tough story versions and test if AI truly "gets" human thinking.

Research Objective

Build a free Python program using open chat AIs (Llama3-8B, Mistral-7B) to test Theory of Mind on normal stories vs automatically-generated tough versions. Show exactly what breaks AI's people-thinking skills, and test if simple fixes like "think step-by-step" actually help.gsb.stanford+1

Research Questions

- Does changing story words, adding confusion, or shuffling sentences break AI thinking?
- Which story types hurt AI most (false beliefs, sarcasm, hints, irony)?
- Do simple prompt tricks make AI better at understanding people?

System Overview

Story Generator: Creates 100 normal + 100 tough ToM stories automatically
AI Tester: Runs Llama/Mistral on stories, scores right/wrong answers
Weakness Finder: Shows which story changes hurt AI performance most
Fix Tester: Tests 3 prompt types (normal, step-by-step, role-play)

Implementation Plan

Phase 1: Download free ToM stories + setup Llama3-8B model (2 days)
Phase 2: Build auto story changer (name swap, add distractors, shuffle) (3 days)
Phase 3: Test 3 AIs on 200 stories, save pass/fail results (3 days)
Phase 4: Test 3 fix prompts on worst stories (2 days)
Phase 5: Make graphs + analyze what breaks AI thinking (3 days)

Evaluation Methodology

Test 3 free models: Llama3-8B, Mistral-7B, Gemma-2B
100 normal stories vs 100 tough versions each
Measure accuracy drop: Normal 80-90% → Tough 30-50%?
Compare 3 prompts: baseline vs chain-of-thought vs "you are psychologist"
Make confusion matrix + drop analysis graphs

Expected Contributions

- Free Python tool to test AI people-thinking (GitHub repo)
- Clear data: "AI drops 50% when stories add 1 distractor sentence"
- Simple fixes that work (or proof popular tricks fail)
- Dataset of 200+ ToM stories for other researchers

Feasibility

30-Day Scope. Laptop + Python only. Free models from HuggingFace (6GB RAM). No GPU needed. 300 lines code total. Datasets on GitHub.

Existing research papers on the topic: https://www.nature.com/articles/s44387-025-00031-9
https://amslaurea.unibo.it/id/eprint/32014/1/huang_xuanqiang_tesi.pdf
https://www.pnas.org/doi/10.1073/pnas.2507080122
https://aclanthology.org/2023.conll-1.25/

Proposed Research Topic 6: Will AI Replace Human Software Engineers? A Deep Learning Code Quality Comparison

Problem Statement

AI tools like GitHub Copilot and Claude now write 40-50% of code in 2026, but companies still hire engineers. Nobody knows exactly which coding tasks AI handles perfectly vs where humans win. Current studies focus on speed, not actual code quality (bugs, maintainability, edge cases). No simple benchmark compares AI vs human engineers on real-world coding problems using objective quality metrics.

Research Objective

Build a Python benchmark that gives 10 real coding problems to both AI models (Llama3-70B, Copilot) and 10 student engineers. Score code on 7 quality metrics (speed, bugs, readability, edge cases, etc.) to show exactly where AI beats humans vs where humans remain essential.

Research Questions

- Does AI code have more bugs than student engineers?
- Which code quality areas does AI completely dominate?
- Can simple prompts make AI match human edge-case handling?

System Overview

Code Problem Generator: 10 real-world problems (API integration, data processing, web scraping)
Dual Tester: AI models + human students solve same problems
Quality Scorer: Automatic tests for bugs, speed, memory, readability (pylint, coverage)
Comparison Dashboard: Bar charts showing AI vs Human across 7 metrics

Implementation Plan

Phase 1: Create 10 coding problems + test cases (2 days)
Phase 2: Run 5 free AI models on problems (prompt engineering) (3 days)

Phase 3: Get 10 students to solve same problems (3 days)
Phase 4: Auto-score all 150 solutions (bugs, speed, pylint score) (2 days)
Phase 5: Make comparison graphs + statistical analysis (3 days)

Evaluation Methodology

10 coding problems x 5 AI models x 10 students = 150 code solutions
7 Quality Metrics:

- Bug rate (unit tests fail %)
- Runtime speed (time command)
- Memory usage (psutil)
- Readability (pylint score)
- Edge case handling (10% test cases)
- Code length (lines of code)
- Documentation quality (docstring coverage)

Expected Contributions

- First objective "AI vs Human Engineer" benchmark with 7 quality metrics
- Clear data: "AI wins speed 3x, humans win edge cases 2x"
- GitHub repo with problems + auto-scorer for other researchers
- Practical guidelines: "Use AI for X, humans for Y"

Feasibility

30-Day Scope. Pure Python. Free AI models (Ollama). Recruit students from college WhatsApp group. Auto-scoring = no manual grading. Runs on any laptop.

What existing papers lack:

Missing Aspect	Why Papers Didn't Do It	Your Advantage
Real-time student engineers (not GitHub mining)	Expensive to recruit	WhatsApp group = free
7 specific quality metrics in **one table**	Scattered analysis	Killer visual for reviewers
2026 models (Llama3-70B, not 2024 GPT-4)	Models evolved	Fresh data
Edge case handling (10% test cases)	Hard to measure	Your unique metric
Prompt engineering ablation	Most used single prompt	Shows AI improvement potential

Existing papers:

1. arXiv 2508.21634 (Aug 2025): AI vs Human on 500k code samples
   - Focus: Static analysis (defects, security, complexity)
   - Used: ChatGPT/DeepSeek vs mined GitHub repos
   - Found: AI simpler but more security bugs
2. MSR 2025: "Can LLMs Generate Higher Quality Code Than Humans?"
   - Focus: GPT-4 vs humans on benchmarks
   - Used: Pylint, Radon, Bandit, TOPSIS ranking
   - Found: GPT-4 beats humans on some metrics
3. arXiv 2501.16857: GPT-4 vs humans (72 tasks)
   - Focus: Standards, security, complexity, functional correctness
   - Found: Humans better standards, GPT-4 more test passes

Tech Stack

- Python + Ollama (Llama3-70B, CodeLlama locally)
- Pytest (auto bug testing), Pylint (readability)
- Pandas/Matplotlib (comparison graphs)
- Google Colab if laptop slow

Proposed Research Topic 4: BERT vs RoBERTa vs DistilBERT - Efficiency Comparison for Laptop Deployment

Problem Statement

BERT, RoBERTa, and DistilBERT are top transformer models for text classification, but students and small teams can't run them easily on normal laptops. Everyone knows RoBERTa is most accurate and DistilBERT is fastest, but no clear 2026 comparison shows exactly how they perform on CPU-only laptops across 5 key metrics (accuracy, speed, memory, battery drain, ease-of-use). Developers need simple guidelines: "Use X model for Y task on 8GB RAM laptop."

Research Objective

Compare BERT-base, RoBERTa-base, and DistilBERT on 3 text tasks (sentiment, spam detection, topic classification) using laptop-only conditions (CPU, 8GB RAM). Measure 5 practical metrics and create "which model for which laptop" guidelines.swiftorial

Research Questions

- Which model runs fastest on CPU-only laptops?
- Does DistilBERT accuracy gap hurt real apps?
- How much battery/memory difference in practice?
- Which model is easiest for beginners to use?

System Overview

Model Tester: Runs 3 models on same 3 datasets
CPU Benchmark: Measures inference speed/memory on laptop CPU

Battery Monitor: Tracks power drain during 1000 predictions
Accuracy Scorer: F1-score on held-out test sets
Ease-of-Use Logger: Lines of code + setup time for each model

Implementation Plan

Phase 1: Setup 3 datasets (IMDB sentiment, spam, 20News) + download models (2 days)
Phase 2: Build CPU benchmark pipeline (HuggingFace pipeline) (2 days)
Phase 3: Run 1000 predictions per model, measure 5 metrics (3 days)
Phase 4: Battery drain test (1 hour continuous inference) (1 day)
Phase 5: Graphs + "model picker" guidelines (3 days)

Evaluation Methodology

3 models × 3 datasets × 5 metrics = 45 data points
Metrics:

- F1-Score: Accuracy on test set
- Inference Speed: Seconds per 100 predictions (CPU)
- Peak RAM: GB used during inference
- Battery Drain: % per hour continuous use
- Setup Time: Minutes to first prediction (beginner)

Expected Contributions

- First 2026 laptop benchmark of BERT family models
- Clear table: Model × Task × 5 metrics
- Practical guidelines: "Use DistilBERT for mobile, RoBERTa for accuracy"
- GitHub repo with CPU benchmark code for students

Feasibility

25-Day Scope. Pure Python + HuggingFace. No GPU needed. Free datasets. Runs on any 8GB RAM laptop. Colab backup if needed.

Tech Stack (100% Free)

- Python + HuggingFace Transformers (BERT/RoBERTa/DistilBERT)
- Datasets: IMDB, SMS Spam, 20NewsGroups (HuggingFace Datasets)
- psutil (RAM/CPU), pandas/matplotlib (analysis)
- CPU-only inference (torch cpu backend)

Why Choose This (IEEE Conference Rocket)
Gap: All comparisons use GPUs. Your angle = CPU-only student laptops (huge practical gap).
Visual: Perfect 3×5 comparison table = reviewer gold.
Target: IEEE Access (90% acceptance) + ICASSP workshops.
Timeless: Model efficiency comparisons = eternal conference favorite.

```text
| Model      | F1 Sentiment | Speed (s/100) | RAM (GB) | Battery/hr | Setup (min) |
|------------|--------------|---------------|----------|------------|-------------|
| BERT-base  | 92.1%        | 18.4          | 3.2      | 12%        | 15          |
| RoBERTa    | 94.3%        | 19.2          | 3.4      | 13%        | 18          |
| **Distil** | **91.8%**    | **9.8**       | **1.9**  | **7%**     | **8**       |
```

Domain: Deep Learning (Transformers + Model Efficiency)

Other papers used GPUs. We will use student laptops.
https://www.swiftorial.com/matchups/nlp_platforms/bert-vs-roberta-vs-distilbert
https://milvus.io/ai-quick-reference/what-differences-in-inference-speed-and-memory-usage-might-you-observe-between-different-sentence-transformer-architectures-for-example-bertbase-vs-distilbert-vs-robertabased-models